



# 匿名函数

## ▼ lambda 表达式

lambda 表达式，也称为 lambda 函数，是在调用或作为函数参数传递的位置处定义匿名函数对象的便捷方法。

lambda 表达式所定义的函数没有明确的函数名称，所以被称为**匿名函数**。

在 C++ 中，匿名函数 (Lambda) 在定义时，会生成一个**闭包对象**，这个对象不仅包含了函数体本身，还包含了捕获的上下文变量。因此，接受匿名函数作为参数时，本质上是接受一个可以调用的对象，而这个对象是由编译器生成的闭包类型实例。

lambda 的主要结构为：定义 + 捕获列表 + 参数列表 + 返回类型 + 函数体 + 立即执行(可选);

```
auto func = []()->{};
// 如果需要在定义完毕后自动立即执行一次，则在分号之前增加一个()
auto func = []()->{}();
```

C++

可以发现 func() 函数并不是直接定义得到的，而是将 lambda 表达式赋值给了 func。

其本质是使用 lambda 表达式生成一个 std::function 类，并将 func 初始化为它的实例。

所以 func 是个函数对象而不是函数本身或函数指针，更不是函数的名字。被赋值给了 func 的函数**依然是匿名函数**，func 只能算作函数的一个接口。

当编译器遇到一个 lambda 表达式时，会在编译时为其生成一个唯一的、匿名的闭包类，这个类包含以下内容：

- **捕获的变量**：如果 lambda 捕获了外部变量，这些变量会作为类的成员变量。
- **operator()**：一个重载的函数调用运算符，包含 lambda 的函数体，通过该重载函数，使闭包类实现了类似于函数的调用方式，实际调用的函数正是该重载函数。

### ▼ 代码示例

```
int factor = 2;
auto lambda = [&](auto&& self, int x) -> int {
```

C++

```

    if (x <= 1) return 1;
    return x * factor * self(self, x - 1); // 递归调用
};

```

#### ▼ 简化版闭包类

C++

```

class __LambdaClosure {
private:
    int& factor; // 捕获的外部变量
public:
    // 构造函数，用于初始化捕获的变量
    __LambdaClosure(int& f) : factor(f) {}

    // 重载的 operator() 实现递归
    template<typename Self>
    int operator()(Self&& self, int x) const {
        if (x <= 1) return 1;
        return x * factor * self(self, x - 1); // 使用捕获的变量进行计算
    }
};

```

这个类的实例就是**闭包对象**，它封装了 Lambda 表达式和其捕获的变量。所以，当 Lambda 表达式赋值给一个变量时，传递的实际上是这个闭包对象。

Lambda 表达式作为参数时，其类型可以是以下方式：

- **函数指针**：当 Lambda 没有捕获外部变量时，它相当于一个普通的函数，可以转换为函数指针传递。
- **闭包对象**：当 Lambda 捕获了外部变量时，它成为一个闭包对象，不再是普通的函数，无法被转换为函数指针，需要通过 `std::function` 或模板参数来接受。

### ▼ 匿名函数的递归

普通函数递归调用时，通过函数名来调用自身。

匿名函数（Lambda）本质上是没名字的，其赋值对象只是一个闭包对象，因此无法在 Lambda 内部通过名称直接调用自身。

所以要实现匿名函数的递归，有两种方式：

- Lambda 通过[&]捕获自身引用，这种方式在生成闭包类时，该类会包含 Lambda 本身作为成员变量，所以必须明确 Lambda 的变量类型。此时如果使用 auto 进行自动推断，则会陷入推断循环，所以这种方式必须通过 `std::function` 显式声明 Lambda 表达式。

```

std::function<int(int)> factorial = [&](int n) -> int {
    if (n <= 1) return 1;
    return n * factorial(n - 1); // 递归调用 Lambda 自身
};

int main() {
    std::cout << factorial(5); // 输出 120
}

```

C++

- Lambda 通过参数传递自身。这种方式不再通过捕获来获取自身引用，所以也就不会将自身作为成员变量。其递归的实现依赖 `operator()`。不需要捕获自身引用，所以可以在递归之前就生成闭包类，而该闭包类中的 `operator()` 函数会再次接受自身引用作为参数，从而实现递归。

```
C++

auto factorial = [](auto&& self, int n) -> int {
    if (n <= 1) return 1;
    return n * self(self, n - 1); // 通过传递自身的引用递归调用
};

int main() {
    std::cout << factorial(factorial, 5); // 输出 120
}
```

- 在 C++23 中提出了新的用法: `auto dfs = [&](this auto&& dfs, int n) -> int{};`

```
C++

auto factorial = [](this auto&& factorial, int n) -> int {
    if (n <= 1) return 1;
    return n * factorial(n - 1); // 通过传递自身的引用递归调用
};

int main() {
    std::cout << factorial(5); // 输出 120
}
```

这种新用法等效于通过 `auto&& self` 传递自身，但是在调用时不再需要显示传入本身。

## ▼ 使用方法

Lambda 的类型是个不具名函数对象，(function object, 或称 functor)。其数据类型为一个函数对象，可以使用 `auto` 来自行推断，也可以使用 `function<T1(T2...)>`，甚至也可以使用函数指针的数据类型 `T1(*) (T2...)`，因为一个没有指定任何捕获的 lambda 函数可以显式转换成一个具有相同声明形式的函数指针。

不过值得一提的是，lambda 函数中如果调用了自己，也就是说发生了自己的递归时，不可以使用 `auto` 来定义，而是要使用 `function<T1(T2...)>`，而且必须使用捕获，因为 lambda 函数需要捕获自身来调用自己。

## 捕获列表

[ ] 内是捕获列表，可以捕获当前作用域内的变量，捕获后可以在 functor 中使用。主要有值传递与引用传递两种方式。

```
C++

[]: 不捕获变量
[var]: 捕获变量名为 var 的变量，方式为值传递
[=]: 捕获作用域内所有变量，方式为值传递
[&var]: 捕获变量名为 var 的变量，方式为引用传递
[&]: 捕获作用域内所有变量，方式为引用传递
```

在捕获外部变量时，匿名函数会生成闭包对象，即捕获的变量和函数一起打包成一个对象，每次调用时可以根据捕获的状态动态调整行为。

## 参数列表

与 function 函数并无区别

## 返回值

可以省略，由编译器自行推断，也可以标明。

## 函数体

与 function 并无区别，在函数体结束后要加分号。